9 Module 9: Main Memory Management

9.1 Compilation System

9.1.1 Introduction to Compilation Systems

- A **compilation system** refers to the collection of **software tools and processes** used to translate **source code** (written in a high-level programming language) into **machine code** that can be executed by a computer's processor.
- **Primary Goal**: Convert human-readable code into a form that the machine can **understand and execute efficiently**.

9.1.2 Execution of a Simple Program: "Hello World" Example

- **Example Program**:

```c
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

  - **Output**: Prints `"Hello World"` on the monitor.

- **Key Question**: How does this program get executed in a computer system?

9.1.3 Phases of the Compilation System

The compilation process consists of **five key phases**, each transforming the program into a different intermediate form before generating the final executable.

9.1.3.1 1. Pre-processing Phase

- **Role**: The **pre-processor** processes directives (statements starting with `#`).
- **Action**:
  - Replaces `#include <stdio.h>` with the **actual content** of the `stdio.h` header file.
  - Expands macros and resolves conditional compilation directives (e.g., `#ifdef`, `#define`).
- **Output**: Generates a **pre-processed file** (`hello.i`).
  - This file contains the **expanded source code** with all included headers and macros.

9.1.3.2 2. Compilation Phase

- **Role**: The **compiler** converts the pre-processed high-level code into **assembly language**.
- **Action**:
  - Performs **syntax and semantic analysis**.
  - Generates **assembly language instructions** (low-level human-readable code).
- **Output**: Produces an **assembly language file** (`hello.s`).
- **Why Not Directly to Machine Code?**
  - **Portability**: High-level code is **platform-independent**.
  - Direct conversion to machine code would **tie the compiled code to a specific architecture**, reducing flexibility.
  - **Intermediate assembly code** allows for **optimization and debugging** before final translation.

9.1.3.3 3. Assembly Phase

- **Role**: The **assembler** converts assembly language into **machine-readable binary object code**.
- **Action**:
  - Translates mnemonics (e.g., `MOV`, `ADD`) into **binary instructions**.
  - Generates **relocatable object code** (contains machine instructions but may have unresolved references).
- **Output**: Produces an **object file** (`hello.o`).
  - This file is **not yet executable**-it lacks linked library functions.

9.1.3.4 4. Linking Phase

- **Role**: The **linker** combines multiple object files and resolves external references.
- **Action**:
  - Links the object code of **library functions** (e.g., `printf` from `stdio.h`) with the program's object file (`hello.o`).
  - Resolves **symbol references** (e.g., function calls, global variables).
  - Generates a **single executable object file**.
- **Output**: Produces an **executable file** (`hello` in Unix-like systems or `hello.exe` in Windows).
  - This file is **ready for execution** but not yet loaded into memory.

9.1.3.5 5. Loading Phase

- **Role**: The **loader** loads the executable into **main memory (RAM)** for execution.
- **Action**:
  - Allocates memory space for the program.
  - Resolves **dynamic linking** (if any shared libraries are used).
  - Transfers control to the **operating system** to begin execution.
- **Output**: The program runs, producing the expected output (e.g., `"Hello World"`).

9.1.4 Why an Intermediate Assembly Phase?

- **Source Code -> Assembly -> Machine Code** (instead of direct conversion) for the following reasons:
  1. **Portability**:
     - High-level code is **platform-independent**.
     - Assembly code acts as an **intermediate representation (IR)** that can be adapted to different architectures.
  2. **Optimization**:
     - Assembly code allows for **manual or compiler-driven optimizations** before final translation.
  3. **Debugging**:
     - Easier to **inspect and debug** assembly code than raw machine code.
  4. **Modular Compilation**:
     - Different parts of a program can be compiled separately and later linked.

9.1.5 Role of Header Files in Compilation

- **Header Files (e.g., `stdio.h`)**:
  - Contain **function declarations**, **macro definitions**, and **data type definitions**.
  - **Not compiled directly**-their content is **inserted into the source file** during pre-processing.
  - The **actual implementation** of functions (e.g., `printf`) is stored in **pre-compiled libraries** (`libc` in Unix-like systems).
- **Linking Process**:

- The linker **resolves references** to these library functions and **combines them** with the program's object code.

9.1.6 Advantages of the Compilation System

1. **Code Optimization**:
   - Compilers perform **optimizations** to improve **execution speed** and **reduce memory usage**.
   - Examples: Loop unrolling, dead code elimination, constant propagation.
2. **Early Error Detection**:
   - Provides **syntax and semantic error messages** during compilation, helping developers **catch bugs early**.
3. **Cross-Platform Compatibility**:
   - The same source code can be **compiled for different hardware platforms** with minimal changes.
   - Example: A C program can run on **x86, ARM, or RISC-V** after recompilation.
4. **Automation of Translation**:
   - Streamlines the **development process** by automating the conversion from source code to executable.
   - Reduces manual effort in writing machine-specific instructions.
5. **Modularity and Reusability**:
   - Supports **separate compilation** of different source files, enabling **modular programming**.
   - Libraries can be **pre-compiled** and reused across multiple programs.

9.1.7 Summary of Key Concepts

- A **compilation system** is a **multi-stage process** that transforms high-level source code into executable machine code.
- **Five Major Phases**:
  1. **Pre-processing** (Handles directives, includes headers).
  2. **Compilation** (Converts to assembly language).
  3. **Assembly** (Converts to machine code).
  4. **Linking** (Combines object files and libraries).
  5. **Loading** (Loads executable into memory for execution).
- **Intermediate assembly phase** enhances **portability, optimization, and debugging**.
- **Header files** provide declarations, while **libraries** provide implementations.
- **Advantages**: Optimization, error detection, cross-platform support, automation, and modularity.

9.1.8 Conclusion

The compilation system is a **fundamental component** of software development, enabling efficient execution of programs by bridging the gap between **human-readable code** and **machine-executable instructions**. Understanding this process is crucial for **debugging, optimization, and cross-platform development**.

9.2 Dynamic Partition Allocation Schemes

9.2.1 1. Introduction to Dynamic Partitioning

- **Definition**: A memory management method where the operating system allocates **exact memory requirements** for each process, unlike fixed partitioning (which uses fixed-size partitions).
- **Key Feature**: Adjusts memory allocation dynamically based on process needs.

9.2.2 2. Four Dynamic Partition Allocation Schemes

Four primary schemes are used to allocate memory blocks to processes:

9.2.2.1 2.1 First-Fit Allocation

- **Definition**: Allocates the **first available hole (free block)** that is **large enough** to accommodate the process.
- **Process**:
  - Scanning begins **from the start of memory**.
  - The first hole that meets the size requirement is selected.
- **Advantages**:
  - Simple and fast (no need to search the entire list).
- **Disadvantages**:
  - May leave **larger leftover holes** in earlier memory regions.
  - Can lead to **external fragmentation** over time.

9.2.2.2 2.2 Best-Fit Allocation

- **Definition**: Allocates the **smallest available hole** that is **large enough** to fit the process.
- **Process**:
  - Requires **searching the entire list** (unless holes are ordered by size).
  - Minimizes **leftover space (wasted memory)**.
- **Advantages**:
  - Produces the **smallest leftover hole**, reducing fragmentation.
- **Disadvantages**:
  - **Slower** due to full-list search.
  - May leave **tiny, unusable fragments** over time.

9.2.2.3 2.3 Next-Fit Allocation

- **Definition**: A variation of first-fit where the search **starts from the last allocation point** instead of the beginning.
- **Process**:
  - Maintains a **pointer** to the last allocated block.
  - Scans memory **circularly** from the last position.
- **Advantages**:
  - More **uniform distribution** of allocations across memory.
  - Slightly **faster than best-fit** (no full-list search).
- **Disadvantages**:
  - May still lead to **larger leftover holes** in some regions.
  - **Less efficient** than best-fit in minimizing waste.

9.2.2.4 2.4 Worst-Fit Allocation

- **Definition**: Allocates the **largest available hole** to the process.
- **Process**:
  - Requires **searching the entire list** to find the largest hole.
  - Leaves the **biggest possible leftover chunk**.
- **Advantages**:
  - Intended to **minimize fragmentation** by keeping large free blocks.
- **Disadvantages**:
  - **Inefficient**-often leaves **very large leftover holes**.
  - **Slower** due to full-list search.
  - May **fail to allocate** smaller processes if large holes are exhausted.

9.2.3 3. Comparative Example: Allocating Processes to Memory Holes

9.2.3.1 3.1 Initial Memory State

- **Available holes (free blocks)** in main memory:
  - 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, 125 KB
- **Processes to allocate** (with memory requirements):
  - 115 KB, 500 KB, 360 KB, 200 KB, 375 KB

9.2.3.2 3.2 Allocation Using First-Fit

Process	Allocated Hole	Leftover Hole
115 KB	300 KB	185 KB
500 KB	600 KB	100 KB
360 KB	750 KB	390 KB
200 KB	350 KB	150 KB
375 KB	390 KB	15 KB

- **Result**:
  - **6 holes** remain (185 KB, 100 KB, 390 KB, 150 KB, 15 KB, 125 KB).
  - **All processes allocated successfully**.

9.2.3.3 3.3 Allocation Using Best-Fit

Process	Allocated Hole	Leftover Hole
115 KB	125 KB	10 KB
500 KB	600 KB	100 KB
360 KB	750 KB	390 KB
200 KB	200 KB	0 KB
375 KB	390 KB	15 KB

- **Result**:
  - **5 holes** remain (10 KB, 100 KB, 390 KB, 15 KB, 300 KB).
  - **All processes allocated successfully**.
  - **Minimal wasted space** compared to other schemes.

9.2.3.4 3.4 Allocation Using Next-Fit

Process	Allocated Hole	Leftover Hole	New Scan Point
115 KB	300 KB	185 KB	After 300 KB
500 KB	600 KB	100 KB	After 600 KB
360 KB	750 KB	390 KB	After 750 KB
200 KB	390 KB	190 KB	After 390 KB
375 KB	**Unallocated**	-	-

- **Result**:
  - **6 holes** remain (185 KB, 100 KB, 190 KB, 350 KB, 200 KB, 125 KB).
  - **375 KB process fails to allocate** (no suitable hole found).

9.2.3.5 3.5 Allocation Using Worst-Fit

Process	Allocated Hole	Leftover Hole
115 KB	750 KB	635 KB
500 KB	635 KB	135 KB
360 KB	600 KB	240 KB
200 KB	350 KB	150 KB
375 KB	**Unallocated**	-

- **Result**:
  - **6 holes** remain (300 KB, 135 KB, 240 KB, 150 KB, 200 KB, 125 KB).
  - **375 KB process fails to allocate** (no hole large enough).

9.2.4 4. Comparison of Allocation Schemes

Scheme	Allocation Strategy	Search Method	Leftover Hole Size	All Processes Allocated?	Fragmentation Tendency
First-Fit	First sufficient hole from start	Linear (from beginning)	Variable	[OK] Yes	Moderate
Best-Fit	Smallest sufficient hole	Full list search	Minimal	[OK] Yes	Low
Next-Fit	First sufficient hole from last allocation	Circular (from last point)	Variable	[X] No (375 KB failed)	Moderate
Worst-Fit	Largest available hole	Full list search	Largest	[X] No (375 KB failed)	High

9.2.4.1 4.1 Key Observations

1. **Best-Fit** produces the **fewest and smallest holes**, making it the **most space-efficient**.
2. **First-Fit** and **Next-Fit** are **faster** but may lead to **more fragmentation**.
3. **Worst-Fit** tends to **waste the most memory** by leaving large leftover holes.
4. **Next-Fit and Worst-Fit failed to allocate all processes**, while **First-Fit and Best-Fit succeeded**.

9.2.5 5. Conclusion: Best Scheme

- **Best-Fit is the superior scheme** among the four because:
  - Minimizes **wasted memory** (smallest leftover holes).
  - Successfully allocates **all processes** in the example.
  - Reduces **external fragmentation** compared to other methods.
- **Trade-off**: Slower due to full-list search, but **better memory utilization** justifies the overhead.

9.3 Dynamic Partition Memory Allocation

9.3.1 1. Introduction to Dynamic Partition Allocation

- **Definition**: A **memory management technique** used by operating systems to **dynamically allocate memory** to processes as they request it.
- **Key Feature**: Partitions are of **variable length and number**, unlike fixed partitioning.
- **Advantage**: Allows **exact memory allocation** (no internal fragmentation within partitions).

9.3.2 2. Example Scenario: Memory Allocation and Release

9.3.2.1 2.1 Initial Memory Setup

- **Total main memory**: 64 MB
- **OS-occupied space**: 8 MB
- **Available memory**: 56 MB

9.3.2.2 2.2 Process Allocation Sequence

1. **Process P1** is allocated first.
2. **Process P2** is allocated next.
3. **Process P3** is allocated afterward.
    - **Result**: A **hole** (unallocated space) of **4 MB** remains after these allocations.

9.3.2.3 2.3 Process Release and Hole Creation

- **Process P2 is released**, creating a **new hole of 14 MB**.
- **Process P4 (8 MB)** is allocated in the **14 MB hole**, leaving a **smaller hole of 6 MB**.

9.3.2.4 2.4 Further Allocation and Fragmentation

- **Process P1 is released**, creating a **20 MB hole**.
- **Process P2 is reallocated** in the **20 MB hole**.
- **Process P5 (8 MB)** arrives but **cannot be allocated** despite sufficient total free memory.

9.3.2.5 2.5 Problem Identified

- **Multiple small holes** exist (each < 8 MB), but **no contiguous block** is large enough for P5.
- **Total free memory > required size**, but **fragmentation prevents allocation**.

9.3.3 3. External Fragmentation

9.3.3.1 3.1 Definition

- **External fragmentation** occurs when **free memory is scattered in small, non-contiguous blocks** ("holes").
- **Cause**: Repeated allocation and release of variable-sized partitions.

9.3.3.2 3.2 Consequences

- **Memory wastage**: Even if total free memory is sufficient, processes may fail to allocate due to lack of contiguous space.
- **System inefficiency**: Over time, fragmentation worsens, reducing usable memory.

9.3.4 4. Solution: Memory Compaction

9.3.4.1 4.1 Definition

- **Compaction** is the process of **moving allocated processes together** to merge free holes into a **larger contiguous block**.

9.3.4.2 4.2 Mechanism

- **Relocates processes** in memory to consolidate free space.
- **Requires dynamic relocation** (processes must be movable at runtime).

9.3.4.3 4.3 Limitations

- **Time-consuming**: Involves copying processes and updating memory references.
- **Processor overhead**: Wastes CPU cycles, impacting system performance.

9.3.5 5. OS Requirements for Dynamic Partitioning

- The OS must maintain **two key data structures**:
  1. **Allocated partitions table**: Tracks memory regions assigned to processes.
  2. **Free partitions table**: Tracks available holes (size and location).

9.3.6 6. Summary of Key Concepts

9.3.6.1 6.1 Dynamic Partitioning

- Allocates memory **on-demand** with **variable-sized partitions**.
- **Pros**: Efficient memory use (no internal fragmentation).
- **Cons**: Leads to **external fragmentation**.

9.3.6.2 6.2 External Fragmentation

- **Problem**: Free memory becomes unusable due to non-contiguity.
- **Impact**: System may reject new processes despite sufficient total memory.

9.3.6.3 6.3 Compaction

- **Solution**: Merges free holes by relocating processes.
- **Trade-off**: Reduces fragmentation but **increases overhead**.

9.4 Fixed Partition Memory Allocation

9.4.1 Introduction to Memory Allocation Schemes

- Memory management systems employ various **memory allocation schemes** to allocate memory to processes.
- Common schemes include:
  - Fixed memory partitioning
  - Segmentation
  - Paging
  - Virtual paging
- This lecture focuses on **fixed partition memory allocation**.

9.4.2 Definition of Fixed Partition Memory Allocation

- **Fixed partition memory allocation** divides **physical memory** into several **static partitions (blocks)** during **system initialization**.
- Once created, these partitions **cannot be modified** (hence the term “fixed”).
- Two primary types based on partition size:
  1. **Fixed-size partitions** (all partitions are equal).
  2. **Variable-size partitions** (partitions have different sizes).

9.4.3 Fixed-Size Partition Memory Allocation

9.4.3.1 Characteristics

- All partitions are of **equal size**.
- Each partition accommodates **only one process** at a time.
- Example: Memory divided into **six partitions**, each of **4MB**.

9.4.3.2 Process Loading Mechanism

- A process is loaded into a partition **large enough** to hold it.
  - The partition may be of **equal or larger size** than the process.
- Example: A **2MB process** can be loaded into a **4MB partition**, leaving **2MB unused**.

9.4.3.3 Advantages

1. **Simplicity**:
   - Easy to implement due to uniform partition sizes.
2. **Low Operating System Overhead**:
   - Fixed number of partitions reduces OS complexity.
3. **Degree of Multiprogramming**:
   - Limited by the **number of partitions** (e.g., 6 partitions -> max 6 processes).
   - Example: **IBM OS/360** used this scheme, known as **MFT (Multiprogramming with a Fixed number of Tasks)**.

9.4.3.4 Disadvantages

1. **Program Size Limitations**:
   - A program **larger than the partition size** cannot be loaded.
   - Example: A **10MB program** cannot fit into a **4MB partition**.
   - **Solution**: Use **overlays** (dividing a program into smaller segments loaded as needed).
2. **Internal Fragmentation**:
   - **Unused memory** within a partition when a process does not fill it entirely.
   - Example: A **2MB process** in a **4MB partition** wastes **2MB**.
   - **Mitigation**:
     - Reducing partition size -> **larger programs may not fit**.
     - Using **variable-size partitions** (more flexible but complex).

9.4.4 Variable-Size Partition Memory Allocation

9.4.4.1 Characteristics

- Partitions have **different sizes**, offering **greater flexibility**.
- Helps accommodate **larger programs** compared to fixed-size partitions.
- Still suffers from **internal fragmentation** but to a lesser extent.

9.4.4.2 Process Assignment Techniques

Two methods to assign processes to partitions:

9.4.4.2.1 1. One Process Queue per Partition

- **Mechanism**:
  - The system maintains a **separate queue for each partition**.
  - Processes are placed in the queue **based on their size**.

  - Example: A **1K partition** has a queue for processes **<=1K**.
- **Advantages**:
  - **Reduces internal fragmentation** (processes matched to appropriately sized partitions).
- **Disadvantages**:
  - **Non-optimal resource usage**:
    * Example: If **Queue 1** has **10 processes** and other queues are **empty**, processes in Queue 1 must wait **even if other partitions are free**.
    * Leads to **underutilization of available partitions**.

9.4.4.2.2 2. Single Queue for All Partitions

- **Mechanism**:
  - All processes enter a **single queue**.
  - The OS assigns processes to **any available partition** that can accommodate them.
- **Advantages**:
  - **More optimal** than per-partition queues (better partition utilization).
- **Disadvantages**:
  - **Higher internal fragmentation** (processes may be placed in larger-than-needed partitions).
  - **Swapping overhead**:
    * If all partitions are occupied, the OS must:
      1. **Swap out** an existing process.
      2. **Swap in** the new process.
    * **Replacement algorithm** determines which process to swap out.

9.4.5 Key Challenges in Fixed Partition Allocation

9.4.5.1 1. Limited Number of Active Processes

- The **number of partitions** is fixed at **system generation time**.
- Restricts the **maximum number of concurrent processes**.

9.4.5.2 2. Inefficient Memory Utilization

- **Small jobs** waste memory due to **internal fragmentation**.
- **Large programs** may not fit, requiring **overlays** (complex and inefficient).

9.4.5.3 3. Swapping Overhead

- When all partitions are full, **swapping** is required:
  - **Swap out**: Remove an existing process from memory.
  - **Swap in**: Load the new process into the freed partition.
- **Scheduling and replacement algorithms** determine efficiency.

9.4.6 Comparison: Fixed vs. Variable Partitioning

Feature	Fixed-Size Partitions	Variable-Size Partitions
Partition Size	All partitions equal.	Partitions of different sizes.
Implementation	Simple.	More complex.
Internal Fragmentation	High (if partition size » process size).	Lower (better size matching).

Feature	Fixed-Size Partitions	Variable-Size Partitions
Flexibility	Low (cannot accommodate large programs easily).	Higher (can fit larger programs).
Overhead	Low OS overhead.	Higher management complexity.
Example System	IBM OS/360 (MFT).	More modern systems with dynamic allocation.

9.4.7 Summary of Key Concepts

9.4.7.1 Fixed Partition Memory Allocation

- **Definition**: Physical memory divided into **static partitions** at system initialization.
- **Types**:
  - **Fixed-size partitions** (equal size, simple but inefficient).
  - **Variable-size partitions** (flexible but complex).
- **Process Assignment**:
  - **Per-partition queues** (reduces fragmentation but suboptimal).
  - **Single queue** (more optimal but higher fragmentation).
- **Challenges**:
  - **Internal fragmentation** (wasted memory within partitions).
  - **Program size limitations** (overlays needed for large programs).
  - **Swapping overhead** (when all partitions are occupied).
- **Historical Example**: **IBM OS/360 (MFT)**.

9.4.7.2 Critical Terms

- **Internal Fragmentation**: Unused memory within a partition after process allocation.
- **Overlays**: Technique to run large programs by dividing them into smaller segments.
- **Swapping**: Moving processes between memory and disk to free up partitions.
- **Degree of Multiprogramming**: Number of processes that can reside in memory simultaneously.
- **MFT (Multiprogramming with a Fixed number of Tasks)**: IBM's fixed-partition memory management system.

9.5 Introduction to Paging

9.5.1 1. Overview of Memory Allocation Schemes

9.5.1.1 1.1 Fixed-Size Partition Memory Allocation

- **Definition**: Memory is divided into **fixed-size blocks (partitions)**, which are allocated to processes.
- **Key Issue**: **Internal fragmentation** occurs when:
  - Allocated memory blocks are **larger** than the requested memory.
  - Leaves **unused space** within the block.

9.5.1.2 1.2 Dynamic Partition Memory Allocation

- **Definition**: Memory is allocated in **variable-size blocks** based on the **exact needs** of processes.
- **Key Issue**: **External fragmentation** occurs when:
  - Free memory is split into **small, non-contiguous blocks**.
  - Makes it difficult to find a **large enough contiguous block** for new processes.

9.5.2 2. Paging: A Solution to Fragmentation Issues

9.5.2.1 2.1 Definition and Purpose

- **Paging** is a memory allocation scheme that:
  - **Reduces internal fragmentation**.
  - **Eliminates external fragmentation**.
- **Key Advantage**: Allows **physical address space** of a process to be **non-contiguous**.

9.5.2.2 2.2 Structure of Paging

- **Physical Memory (RAM)** is divided into **fixed-size blocks** called **frames**.
- **Logical Memory (Process Address Space)** is divided into **fixed-size blocks** called **pages**.
- **Frame and Page Size**:
  - Both are of **equal size** (typically a **power of two**, e.g., 4 KB, 8 KB).
  - Each frame holds **one page** of logical memory.

9.5.2.3 2.3 Memory Allocation Process

- To run a program of size **n pages**, the OS must:
  1. Find **n free frames** in physical memory.
  2. Load the program's pages into these frames.

9.5.2.3.1 Example: Process Allocation

- **Scenario**:
  - **Processes**:
    - A (4 pages), B (3 pages), C (4 pages), D (5 pages).
  - **Total Frames in Memory**: 15.
- **Allocation Steps**:
  1. **Process A** (4 pages) -> Allocated to frames 1-4.
  2. **Process B** (3 pages) -> Allocated to frames 5-7.
  3. **Process C** (4 pages) -> Allocated to frames 8-11.
  4. **Process D** (5 pages) -> **Cannot be allocated** (only 4 frames left).
  5. **Process B terminates** -> Frames 5-7 freed.
  6. **Process D** now allocated to frames 4-6 (3 pages) + frames 11-12 (2 pages).

9.5.3 3. Implementation of Paging

9.5.3.1 3.1 Requirements

1. **Free Frame Tracking**: OS must track which frames are free.
2. **Page Table**: Maps **logical addresses** to **physical addresses**.

9.5.3.2 3.2 Address Translation Mechanism

- **Logical Address Structure**:
  - Divided into:
    - **Page Number (P)**: Index into the page table.
    - **Page Offset (d)**: Displacement within the page.
- **Physical Address Calculation**:
  1. **Page Table Lookup**:
     - Use **P** to find the **frame number** in the page table.

2. **Offset Addition**:
   - Add **d** to the frame's base address -> **Physical Address**.

9.5.3.2.1 Formula for Address Bits

- **Logical Address Space**: 2^m (total addresses).
- **Page Size**: 2^n (addresses per page).
- **Page Offset (d)**: **n bits** (to address within a page).
- **Page Number (P)**: **(m - n) bits** (to index the page table).

9.5.4 4. Address Translation Architecture

9.5.4.1 4.1 Page Table Structure

- **Per-Process Page Table**:
  - Each process has its own page table.
  - **Entries**: Map **logical page numbers** to **physical frame numbers**.
- **CPU-Generated Logical Address**:
  - Split into **P (page number)** and **d (offset)**.
  - **P** indexes the page table -> **frame number**.
  - **d** is added to the frame number -> **physical address**.

9.5.4.2 4.2 Example 1: Page Table Mapping

- **Process Logical Memory**: 4 pages (P0-P3).
- **Page Table Entries**: | Page Number (P) | Frame Number | |—————|————| | 0 | 1 | | 1 | 4 | | 2 | 3 | | 3 | 7 |

9.5.4.3 4.3 Example 2: Logical to Physical Address Translation

- **Logical Memory**:
  - 16 locations (A-P), **4-bit addressing**.
  - **Pages**: P0-P3 (each page = 4 locations).
  - **Page Size**: 4 -> **2-bit offset (d)**.
  - **Page Number (P)**: 2 bits (since 2^2 = 4 pages).
- **Physical Memory**:
  - 32 locations, **5-bit addressing**.
  - **Frame Size**: 4 -> **8 frames (f0-f7)**.
  - **Frame Number**: 3 bits (since 2^3 = 8 frames).
- **Translation Steps**:
  1. **Logical Address**: 5 (binary: `0101`).
     - **P**: `01` (page 1).
     - **d**: `01` (offset 1).
  2. **Page Table Lookup**:
     - Index 1 -> Frame 6 (`110` in binary).
  3. **Physical Address**:
     - Append **d** -> `11001` (25 in decimal).
     - **Frame 6**, offset 1 -> Points to **character F**.

9.5.5 5. Page Size Considerations

9.5.5.1 5.1 Impact of Page Size

Factor	Small Pages	Large Pages
Internal Fragmentation	Low (less wasted space per page)	High (more wasted space per page)
Page Table Size	Large (more pages -> more entries)	Small (fewer pages -> fewer entries)
Disk I/O Efficiency	Low (more pages -> more I/O operations)	High (fewer pages -> fewer I/O operations)

9.5.5.2 5.2 Optimal Page Size

- **Typical Range**: **4 KB to 8 KB**.
- **Trade-offs**:
  - **Small Pages**:
    - Reduce internal fragmentation.
    - Increase page table overhead.
    - More frequent disk I/O.
  - **Large Pages**:
    - Reduce page table size.
    - Increase internal fragmentation.
    - Improve disk I/O efficiency (larger transfers).

9.5.5.3 5.3 Additional OS Overhead

- **Frame Table**: OS maintains a table to track **allocated/free frames** in physical memory.

9.5.6 6. Summary of Paging

9.5.6.1 6.1 Key Features

- **Dynamic Allocation**: Allows non-contiguous physical memory usage.
- **Elimination of External Fragmentation**: No need for contiguous blocks.
- **Internal Fragmentation**: Still possible (depends on page size).
- **Address Translation**: Uses **page tables** to map logical -> physical addresses.

9.5.6.2 6.2 Advantages Over Fixed/Dynamic Partitioning

Scheme	Internal Fragmentation	External Fragmentation	Contiguity Requirement
Fixed-Size Partition	High	None	Yes
Dynamic Partition	Low	High	Yes
Paging	**Low (depends on page size)**	**None**	**No**

9.5.6.3 6.3 Optimal Page Size Selection

- Depends on:
  - **System workload** (memory-intensive vs. I/O-intensive).
  - **Hardware constraints** (e.g., TLB size, disk block size).
  - **Trade-off analysis** between fragmentation, page table size, and I/O efficiency.

9.6 Introduction to Segmentation

9.6.1 1. Overview of Segmentation

- **Definition**: Segmentation is a **memory management technique** that supports the **user's view of memory** by dividing a process into **logical, variable-sized segments** (e.g., main program, functions, arrays, variables, symbol tables).
- **Key Distinction from Paging**:
  - **Paging** divides logical space into **fixed-size pages** loaded into **equal-sized frames** in physical memory.
  - **Segmentation** divides logical space into **variable-sized segments** loaded into **non-contiguous memory locations** in physical memory.
- **Alignment with Programming**:
  - Segmentation reflects the **modular structure of programs**, where a program is a **collection of modules/segments** (e.g., functions, data structures).
  - Unlike paging, which is **hardware-oriented**, segmentation is **user-oriented** and aligns with how programmers organize code and data.

9.6.2 2. Structure of Segments in Memory

9.6.2.1 2.1. Segment Characteristics

- **Variable Size**: Each segment has a **different size** based on its content (e.g., a large array vs. a small function).
- **Contiguity Within Segments**:
  - **Code/data within a single segment** is stored in **contiguous memory locations**.
  - **Different segments of the same process** can be loaded into **non-contiguous memory locations**.
- **Dynamic Loading**:
  - Segments are loaded **on-demand** (e.g., the **main program** loads first; a **function segment** loads only when called).

9.6.2.2 2.2. Example of Segment Allocation

- **Scenario**:
  1. The **main program segment** is loaded into memory first.
  2. When a **function (e.g., `read_data`)** is called, its corresponding segment is loaded into a **different memory location**.
- **Implication**: Segments do not need to occupy adjacent memory blocks, but **each segment itself is contiguous**.

9.6.3 3. Segment Table

9.6.3.1 3.1. Purpose

- Analogous to the **page table** in paging, the **segment table** maps **logical segments** to **physical memory locations**.
- Each process has its own **segment table**.

9.6.3.2 3.2. Structure of a Segment Table Entry Each entry contains: 1. **Base (Starting Address)**: - The **physical memory address** where the segment begins. - Example: Segment 0 starts at **address 1000**. 2. **Limit (Segment Length)**: - The **size of the segment** (number of memory locations allocated). - Example: Segment 0 has a limit of **100**, so its last address is **1099 (1000 + 99)**.

9.6.3.3 3.3. Visual Representation

- **Segment Table Example**: | Segment Number | Base | Limit | |——————-|——-|——-| | 0 | 1000 | 100 | | 1 | 2000 | 50 | | 2 | 5000 | 200 | | 3 | 8000 | 150 |
- **Physical Memory Allocation**:
  - Segment 0: **1000-1099**
  - Segment 1: **2000-2049**
  - Segment 2: **5000-5199**
  - Segment 3: **8000-8149**

9.6.4 4. Logical to Physical Address Translation

9.6.4.1 4.1. Logical Address Format

- Represented as a **tuple**: `(segment_number, offset)`
  - **segment_number**: Identifies the segment (e.g., 0 for main program, 1 for a function).
  - **offset**: Displacement within the segment (must be <= segment limit).

9.6.4.2 4.2. Translation Process

1. **Extract Segment Number**:
   - Use the **segment_number** to index into the **segment table**.
2. **Check Validity**:
   - Compare the **offset** with the **limit** from the segment table:
     - If **offset < limit** -> **Valid address**.
     - If **offset >= limit** -> **Trap (addressing error)**.
3. **Compute Physical Address**:
   - **Physical Address = Base + Offset**
   - Example:
     - Logical address: `(0, 50)`
     - Segment 0: Base = 1000, Limit = 100
     - **50 < 100** -> Valid
     - Physical address = **1000 + 50 = 1050**

9.6.4.3 4.3. Error Handling

- If the **segment_number** is **invalid** (e.g., exceeds the number of segments), a **trap** is generated.
- If the **offset** exceeds the **limit**, a **segmentation fault** occurs.

9.6.5 5. Hardware Support for Segmentation

9.6.5.1 5.1. Segment-Table Base Register (STBR)

- **Purpose**: Points to the **starting address of the segment table** in memory.
- **Function**: Used by the CPU to locate the segment table for the current process.

9.6.5.2 5.2. Segment-Table Length Register (STLR)

- **Purpose**: Stores the **number of segments** used by the program.
- **Function**: Ensures the **segment_number** in a logical address is **valid** (i.e., `segment_number < STLR`).

9.6.5.3 5.3. Address Translation Workflow

1. CPU fetches the **logical address** `(s, offset)`.
2. Checks if **s < STLR** (valid segment number).
3. Uses **STBR** to locate the segment table.
4. Retrieves **Base** and **Limit** for segment **s**.
5. Validates **offset < Limit**.
6. Computes **Physical Address = Base + Offset**.

9.6.6 6. Advantages of Segmentation

1. **Modularity**:
   - Reflects the **logical structure of programs** (e.g., code, data, stack as separate segments).
2. **Protection and Sharing**:
   - **Protection**: Each segment can have **different access permissions** (e.g., read-only for code, read-write for data).
   - **Sharing**: Multiple processes can **share segments** (e.g., shared libraries).
3. **Dynamic Growth**:
   - Segments can **grow or shrink** as needed (e.g., a stack segment expanding).
4. **Efficient Memory Usage**:
   - Only **required segments** are loaded into memory (reduces memory overhead).

9.6.7 7. Challenges of Segmentation

1. **External Fragmentation**:
   - **Problem**: Variable-sized segments lead to **gaps in memory** that may be too small for new segments.
   - **Solution**: Requires **compaction** or **advanced allocation strategies** (e.g., best-fit, worst-fit).
2. **Complexity**:
   - **Overhead**: Managing **variable-sized segments** is more complex than fixed-size pages.
   - **Hardware Support**: Requires **additional registers (STBR, STLR)** and **segment table lookups**.
3. **Memory Allocation**:
   - Finding **contiguous blocks** for large segments can be difficult in a fragmented memory.

9.6.8 8. Comparison with Paging

Feature	**Segmentation**	**Paging**
Unit of Division	Variable-sized segments (logical)	Fixed-size pages (hardware-oriented)
Memory Allocation	Non-contiguous between segments	Non-contiguous (frames can be anywhere)
Fragmentation	External fragmentation	Internal fragmentation (fixed page size)
User View	Aligns with program structure	Transparent to the user
Sharing	Easier (logical units)	Harder (requires identical page mapping)
Protection	Fine-grained (per segment)	Coarse-grained (per page)

9.6.9 9. Summary of Key Concepts

- Segmentation divides a process into **logical, variable-sized segments** (e.g., code, data, stack).
- **Segment table** maps logical segments to physical memory using **base and limit** values.
- **Logical address** `(segment_number, offset)` is translated to a **physical address** via the segment table.
- **Hardware support** (STBR, STLR) ensures efficient address translation.

- **Advantages**: Modularity, protection, sharing, dynamic growth.
- **Challenges**: External fragmentation, complexity in memory management.
- **Comparison with Paging**: Segmentation is **user-centric**, while paging is **hardware-centric**.

9.7 Introduction

9.7.1 1. Overview of Memory Management in Operating Systems

- **Primary Objective of an OS**:
  - One of the core functions of an **operating system (OS)** is **memory management**.
  - Efficient memory management ensures optimal system performance and stability.
- **Types of Memory**:
  - **Main Memory (RAM)**:
    - Fast, volatile memory directly accessible by the CPU.
    - Primary focus of this module.
  - **Secondary Memory (Disk/SSD)**:
    - Non-volatile, slower storage used for long-term data retention.
    - Not the primary focus of this session.

9.7.2 2. Role of the Operating System in Main Memory Management

The OS performs critical functions to manage **main memory (RAM)**, including:

1. **Memory Allocation**:
   - Assigning blocks of main memory to processes when they are loaded for execution.
   - Ensures processes have the necessary memory resources to run.
2. **Memory Deallocation**:
   - Reclaiming memory blocks when they are no longer needed by a process.
   - Prevents **memory leaks** (discussed later).
3. **Memory Protection**:
   - Ensuring processes do not access memory allocated to other processes.
   - Prevents unauthorized or accidental memory corruption.
4. **Memory Paging and Segmentation**:
   - Techniques to manage memory efficiently by dividing it into fixed-size (**paging**) or variable-size (**segmentation**) blocks.
5. **Memory Scheduling**:
   - Deciding which processes should reside in memory at a given time.
   - Balances system load and ensures fair resource distribution.

9.7.3 3. Definition of Memory Management

- **Memory Management**:
  - The **process of controlling and coordinating main memory activities** in a computer system.
  - Essential for the **proper functioning of the OS and applications**.
- **Key Requirement for Process Execution**:
  - A **process** (a program in execution) **must reside in main memory** to run.
  - Memory management ensures processes are loaded, executed, and removed efficiently.

9.7.4 4. Memory Allocation and Deallocation

9.7.4.1 4.1 Memory Allocation

- The OS assigns memory blocks to processes when they are:
  - **Loaded for execution**.
  - **Requesting additional memory** (e.g., dynamic memory allocation in programs).

9.7.4.2 4.2 Memory Deallocation

- The OS reclaims memory when:
  - A process **terminates**.
  - Memory is **explicitly freed** by the program (e.g., `free()` in C, garbage collection in Java).
- **Failure to deallocate** leads to **memory leaks**.

9.7.5 5. Memory Leaks: Causes and Consequences

9.7.5.1 5.1 Definition

- A **memory leak** occurs when a process **loses track of allocated memory blocks** and fails to release them after use.
- The memory remains **unavailable for reuse**, even though it is no longer needed.

9.7.5.2 5.2 Causes

- **Programming Errors**:
  - Forgetting to call `free()` (in C/C++) or relying on manual memory management.
  - Dangling pointers or lost references to allocated memory.
- **Improper Resource Handling**:
  - Not closing file handles, network sockets, or other system resources that consume memory.

9.7.5.3 5.3 Consequences

- **Performance Degradation**:
  - Accumulation of unused memory blocks reduces available RAM.
  - Forces the system to rely more on **virtual memory (disk swapping)**, slowing down execution.
- **System Instability**:
  - Severe memory leaks can lead to:
    - **Out-of-memory (OOM) errors**.
    - **System crashes** if critical processes cannot allocate memory.
  - Applications may **freeze or terminate unexpectedly**.

9.7.6 6. Characteristics of Effective Memory Management

A **good memory management system** should exhibit the following properties:

9.7.6.1 6.1 Improves Degree of Multiprogramming

- **Degree of Multiprogramming**:
  - The number of processes that can **reside in main memory simultaneously**.
- **Impact on System Efficiency**:
  - Higher multiprogramming allows **better CPU utilization** (reduces idle time).
  - Ensures a **steady supply of ready processes** for the CPU to execute.

9.7.6.2 6.2 Enhances Efficiency

- **Optimal Memory Allocation**:
  - Allocates memory based on process requirements without **wastage (fragmentation)**.
- **Reduces Overhead**:
  - Minimizes time spent on **allocation/deallocation operations**.
- **Minimizes Disk Accesses**:
  - In **virtual memory systems**, reduces the need for **swapping** (moving data between RAM and disk).

9.7.6.3 6.3 Boosts Performance

- **Faster Memory Access**:
  - Optimizes **cache usage** and **memory hierarchy** for quicker data retrieval.
- **Reduces Latency**:
  - Ensures frequently used data remains in **main memory** rather than being paged out to disk.

9.7.6.4 6.4 Ensures Reliability

- **Correct Memory Allocation**:
  - Guarantees processes receive the **right amount of memory** without errors.
- **Prevents Memory Corruption**:
  - Ensures processes **cannot access memory outside their allocated space**.

9.7.6.5 6.5 Provides Security

- **Memory Protection Mechanisms**:
  - Prevents **unauthorized access** to memory regions (e.g., one process reading another's data).
  - Implements **access control** (read/write/execute permissions).

9.7.6.6 6.6 Supports Scalability

- **Adapts to Varying Workloads**:
  - Handles **dynamic memory demands** (e.g., sudden spikes in process creation).
  - Works efficiently across different **system configurations** (e.g., low-memory vs. high-memory machines).

9.7.6.7 6.7 Ensures Ease of Use and Maintenance

- **User-Friendly Interfaces**:
  - Provides **clear APIs** for memory allocation/deallocation (e.g., `malloc()`, `free()` in C).
- **Diagnostic Tools**:
  - Includes utilities for:
    - **Monitoring memory usage** (e.g., `top`, `htop`, `vmstat` in Linux).
    - **Detecting memory leaks** (e.g., Valgrind, AddressSanitizer).
  - Simplifies **debugging and optimization**.

9.7.7 7. Process Swapping and Memory Tracking

9.7.7.1 7.1 Process Swapping

- **Moving Processes Between Main Memory and Disk**:
  - In systems with **limited RAM**, the OS may **swap out** inactive processes to disk.
  - **Swapped-in processes** are reload into memory when needed.

- **Impact on Performance**:
  - **Swapping is slow** (disk I/O is orders of magnitude slower than RAM access).
  - Excessive swapping leads to **thrashing** (system spends more time swapping than executing).

9.7.7.2 7.2 Memory Tracking The OS must maintain records of: 1. **Allocated Memory Blocks**: - Which processes own which memory regions. - Size and location of each allocation. 2. **Free Memory Blocks**: - Available memory regions for new allocations. - Techniques like **bitmaps** or **linked lists** are used to track free space.

9.7.8 8. Summary of Key Points

This session covered: 1. **Objective of Memory Management**: - A core OS function to manage **main memory (RAM)** efficiently. 2. **Key Functions of the OS**: - Allocation, deallocation, protection, paging/segmentation, and scheduling. 3. **Memory Leaks**: - Causes (programming errors), consequences (performance degradation, crashes). 4. **Features of Effective Memory Management**: - **Multiprogramming**, **efficiency**, **performance**, **reliability**, **security**, **scalability**, and **usability**. 5. **Process Swapping and Memory Tracking**: - Moving processes between RAM and disk, tracking allocated/free memory.

End of Notes

9.8 Main Memory Management Requirements

9.8.1 Introduction to Main Memory Management

- **Purpose**: Before exploring memory management mechanisms, it is essential to understand the fundamental **requirements** of main memory management.
- **Context**:
  - A program must reside in **main memory** to execute.
  - Processes begin in the **job queue** (on disk) and are loaded into main memory for execution.
  - The **main memory management system** determines where a program is loaded based on available free blocks.

9.8.2 Key Questions Addressed

1. **Where will a program be loaded in main memory?**
   - Unknown to the programmer; determined dynamically by the memory management system.
2. **Will a program occupy the same memory block across multiple executions?**
   - **No**. The location depends on available free space at load time.
   - Example: A process (e.g., `P1`) swapped out to accommodate another process (e.g., `P2`) may return to a **different memory location**.

9.8.3 Logical vs. Physical Addresses

9.8.3.1 Definitions

- **Logical Address (Virtual Address)**:
  - Generated by the **CPU**.
  - Represents the address space from the processor's perspective (e.g., `0` to `max`).
  - **Virtual**: Does not correspond to a real memory location.
- **Physical Address (Real Address)**:
  - Actual location in **main memory**.
  - Must be derived from the logical address via **address binding**.

9.8.3.2 Address Binding

- **Definition**: Mapping from **logical address space** to **physical address space**.
- **Stages of Binding**: Binding can occur at **three distinct stages**:
  1. **Compile Time**:
     - **Condition**: Memory location must be known **in advance**.
     - **Output**: **Absolute code** (fixed memory references).
       - Example: If a program is compiled to load at location `1000`, all references are relative to `1000`.
     - **Limitation**: If the starting address changes, the **entire program must be recompiled**.
  2. **Load Time**:
     - **Condition**: Memory location is **unknown at compile time**.
     - **Output**: **Relocatable code** (adjustable memory references).
       - Final binding occurs when the program is **loaded into memory**.
     - **Advantage**: No recompilation needed if the starting address changes; only **reloading** is required.
  3. **Execution (Run) Time**:
     - **Condition**: Process may be **swapped in/out** of memory during execution.
     - **Mechanism**: Binding is **delayed until runtime**.
     - **Requirement**: **Hardware support** (e.g., **Memory Management Unit, MMU**).
     - **Example**: Dynamic relocation registers (e.g., **base and limit registers**).

9.8.4 Memory Protection

9.8.4.1 Problem Statement

- Processes are allocated **separate memory addresses**.
  - Example:
    - Process `P0`: Addresses `2500` to `3499` (size = `1000` locations).
    - Process `P1`: Starts at `3500`.
- **Risk**: A process (e.g., `P1`) may **accidentally or maliciously** access/modify another process's memory (e.g., `P2`).
  - **Consequences**: Security breaches, data corruption, system crashes.

9.8.4.2 Solution: Hardware-Based Protection

- **Mechanism**: Use of **base and limit registers**.
  - **Base Register**:
    - Stores the **smallest legal physical address** (starting address of the process).
  - **Limit Register**:
    - Specifies the **size of the process's address space** (e.g., `1000` for `P0`).
- **Operation**:
  1. CPU generates a **logical address**.
  2. The address is **compared** against the **base + limit** range.
  3. If the address is **outside the range**, a **fatal error** (trap) is triggered, and the **OS is notified**.
- **Key Points**:
  - **Privileged Instructions**: Base/limit registers are loaded in **kernel mode** (only the OS can modify them).
  - **OS Access**: The OS has **unrestricted access** to both **user and OS memory**.

9.8.5 Memory Sharing

9.8.5.1 Motivation

- **Efficiency**: Avoid duplicating identical code/data across processes.
  - Example: Multiple processes executing the **same program** (e.g., a shared library).
- **Collaboration**: Cooperating processes may need to **share data structures** (e.g., shared memory in IPC).

9.8.5.2 Requirements

- **Controlled Access**: The memory management system must:
  1. Allow **shared access** to specific memory regions.
  2. **Preserve protection** (e.g., prevent unauthorized modifications).
- **Implementation**:
  - **Read-Only Sharing**: Code segments can be shared as **read-only** to prevent modifications.
  - **Cooperative Sharing**: Data segments can be shared with **controlled write access**.

9.8.6 Logical vs. Physical Organization

9.8.6.1 Logical Organization (Programmer's View)

- **Imaginary**: Represents how programmers **conceptualize** memory.
  - **Modularity**:
    - Programs are written in **modules** (independent units).
    - **Advantages**:
      1. Modules can be **compiled independently**.
      2. **Different protection levels** can be assigned (e.g., read-only, execute-only).
      3. **Sharing**: Modules can be shared across processes.
- **Address Space**: Logical addresses are **virtual** and must be mapped to physical addresses.

9.8.6.2 Physical Organization (Hardware View)

- **Main Memory**:
  - **Structure**: Linear/one-dimensional array.
  - **Access Unit**:
    - **Byte-addressable**: Each location stores a **byte**.
    - **Word-addressable**: Each location stores a **word**.
  - **Characteristics**:
    - **Fast access**, **high cost**, **volatile**, **smaller capacity**.
- **Secondary Memory**:
  - **Structure**: Similar to main memory (linear array).
  - **Characteristics**:
    - **Slower access**, **low cost**, **non-volatile**, **larger capacity**.
- **Management**:
  - **Not programmable**: Requires a **Memory Management Unit (MMU)** to handle translations and protection.

9.8.7 Summary of Main Memory Management Requirements

The lecture outlined **five core requirements** for main memory management:

1. **Relocation**:
   - Dynamic placement of processes in memory.
   - Address binding at **compile, load, or execution time**.
2. **Protection**:
   - Prevent unauthorized access/modification of memory.

- Implemented via **base/limit registers** and **hardware checks**.

3. **Sharing**:
   - Enable controlled access to shared code/data.
   - Avoid redundancy (e.g., single copy of shared libraries).
4. **Logical Organization**:
   - Programmer's view: **modular, virtual address space**.
   - Supports independent compilation and protection levels.
5. **Physical Organization**:
   - Hardware view: **linear memory array** with distinct access properties.
   - Managed by the **MMU** (not directly by programmers).

9.9 Memory Management Unit

9.9.1 1. Introduction to Memory Management System (MMS)

- A **Memory Management System (MMS)** is a critical component of an **operating system (OS)**.
- **Primary responsibilities**:
  - Controlling and coordinating **allocation** and **de-allocation** of memory resources.
  - Ensuring **efficient and effective utilization** of available memory.
  - Providing a **secure and reliable** environment for running processes and applications.

9.9.2 2. Hardware Support for Memory Management

- Memory management requires **hardware assistance** to function effectively.
- The **Memory Management Unit (MMU)** is the key hardware component responsible for **address translation** and **memory protection**.

9.9.3 3. Types of Addresses in Memory Management

9.9.3.1 3.1 Logical Address (Virtual Address)

- **Definition**: An address generated by the **CPU** during program execution.
- **Alternative name**: Virtual address.
- **Characteristics**:
  - Represents a **memory location** within a **process's address space** (the range of memory addresses accessible to the process).
  - Generated by the **CPU's Arithmetic Logic Unit (ALU)** as instructions are executed.
  - **Relative** to the process's **logical memory space** (does **not** directly correspond to physical memory locations).
  - The **process** operates under the illusion that it is running in a **contiguous memory space** starting from **0** to **max**.

9.9.3.2 3.2 Physical Address

- **Definition**: The **actual location** of data in **physical memory (RAM)**.
- **Characteristics**:
  - Represents the **exact hardware location** where data is stored.
  - The **MMU** translates **logical addresses** into **physical addresses** before memory access.

9.9.4 4. Role of the Memory Management Unit (MMU)

- **Primary function**: **Maps logical addresses to physical addresses**.

- **Key operations**:
  - **Address translation**: Converts CPU-generated logical addresses into physical memory locations.
  - **Memory protection**: Ensures processes **do not access memory outside their allocated segment**.
- **User program perspective**:
  - Only **logical addresses** are visible to the program.
  - The **physical addresses** remain **hidden** from the user process.

9.9.5 5. MMU Registers for Memory Management

The MMU uses **three key registers** to manage memory allocation and protection:

9.9.5.1 5.1 Base Register (Base Address Register)

- **Definition**: Holds the **starting address (base address)** of the **memory segment** allocated to a process.
- **Function**:
  - When a program executes, the **base register** specifies the **beginning of the memory segment** assigned to it.
  - During **address translation**, the MMU **adds the base register value** to the **logical address** to compute the **physical address**.
- **Formula**:

```
Physical Address = Base Register Value + Logical Address
```

9.9.5.2 5.2 Limit Register

- **Definition**: Specifies the **size (length)** of the memory segment allocated to a process.
- **Function**:
  - Ensures that a process **does not access memory beyond its allocated segment**.
  - The MMU **compares** the logical address against the **limit register** to enforce **memory protection**.
  - If the logical address **exceeds the limit**, the MMU **generates a trap (interrupt)** to the OS, indicating an **address violation**.

9.9.5.3 5.3 Relocation Register

- **Definition**: Used in **dynamic relocation** to adjust logical addresses to physical addresses.
- **Function**:
  - The **relocation register** contains a **base value (R)** that is **added to every logical address** generated by the CPU.
  - The **physical address space** is calculated as:
    `Physical Address Space = R + 0 to R + Max`
    where:
    - **R** = Relocation register value
    - **Max** = Maximum logical address

9.9.6 6. Address Translation Process

9.9.6.1 6.1 Dynamic Relocation Using Relocation Register

- **Process**:

1. The **CPU generates a logical address** (e.g., **346**).

2. The **MMU adds the relocation register value** (e.g., **14,000**) to the logical address.
3. The **resulting physical address** is computed (e.g., **14,346**).

- **Example**:

```
Logical Address = 346
Relocation Register = 14,000
Physical Address = 14,000 + 346 = 14,346
```

9.9.6.2 6.2 Address Validation Using Base and Limit Registers

- **Process**:
  1. The **CPU generates a logical address** (e.g., **1,200**).
  2. The **MMU checks** if the address is **>= base register value** (e.g., **1,000**).
     - If **no**, the address is **invalid** (trap generated).
  3. The **MMU checks** if the address is **<= (base + limit)** (e.g., **1,000 + 499 = 1,499**).
     - If **yes**, the address is **valid**.
     - If **no**, the address is **invalid** (trap generated).
  4. If **valid**, the **physical address** is computed as:
     `Physical Address = Base Register + Logical Address`

9.9.6.2.1 Example 1: Valid Address

- **Given**:
  - Base Register = **1,000**
  - Limit Register = **499**
  - Logical Address = **1,200**
- **Check**:
  - **1,200 >= 1,000** -> **True**
  - **1,200 <= (1,000 + 499 = 1,499)** -> **True**
- **Result**: **Valid address** -> Physical address = **1,000 + 1,200 = 2,200** (assuming base is added; exact computation depends on MMU implementation).

9.9.6.2.2 Example 2: Invalid Address (Trap Generated)

- **Given**:
  - Base Register = **1,000**
  - Limit Register = **499**
  - Logical Address = **1,800**
- **Check**:
  - **1,800 >= 1,000** -> **True**
  - **1,800 <= 1,499** -> **False**
- **Result**: **Invalid address** -> MMU **generates a trap (interrupt)** to the OS, indicating an **address error**.

9.9.7 7. Summary of Key Concepts

Concept	Description
Logical Address	Generated by CPU; relative to process's address space (0 to Max).
Physical Address	Actual location in RAM; computed by MMU.

Concept	Description
Base Register	Stores the **starting address** of a process's memory segment.
Limit Register	Defines the **size** of the allocated memory segment.
Relocation Register	Used in **dynamic relocation** to adjust logical to physical addresses.
MMU Function	Translates logical -> physical addresses; enforces memory protection.
Trap (Interrupt)	Generated when a process accesses memory outside its allocated segment.

9.9.7.1 7.1 Key Takeaways

1. The **MMU** is essential for **address translation** and **memory protection**.
2. **Base and limit registers** ensure processes **stay within their allocated memory**.
3. **Relocation registers** enable **dynamic memory allocation** by adjusting logical addresses.
4. **Invalid memory access** results in a **trap**, handled by the OS.
5. **User programs** operate in a **virtual address space**, unaware of physical memory locations.

9.10 Overlays

9.10.1 Introduction to Overlays

9.10.1.1 Background: Fixed Partition Memory Allocation

- **Fixed Partition Allocation**:
  - Memory is divided into fixed-size or variable-size partitions during system initialization.
  - **Drawback**: If a process's memory requirement exceeds the partition size, memory allocation becomes impossible.

9.10.1.2 Purpose of Overlays

- **Definition**: Overlays are a memory management technique used to efficiently manage limited memory resources.
- **Functionality**:
  - Allows programs to be larger than the available physical memory.
  - Divides programs into smaller modules (overlays) and loads only the necessary modules into memory at any given time.

9.10.2 Understanding Overlays Through an Example: Assembler

9.10.2.1 Overview of an Assembler

- **Definition**: A software tool that translates assembly language instructions into machine code.
- **Two-Pass Process**:
  1. **Pass 1**:
     - Reads the entire assembly language program line by line.
     - Tasks:
       * Identification of labels and symbols.
       * Opcode processing.

        - Generating a symbol table.
        - Handling assembler directives.
    - **Note**: Does not generate machine code; focuses on analysis and gathering information for Pass 2.
2. **Pass 2**:
    - Uses information from Pass 1 to generate machine code.
    - Translates instructions into machine language line by line.

9.10.2.2 Memory Requirements of the Assembler

- **Code Sizes**:
    - Pass 1: 70 KB
    - Pass 2: 80 KB
    - Symbol table: 25 KB
    - Common routines: 25 KB
- **Total Assembler Size**: 200 KB

9.10.2.3 Problem Statement

- **Main Memory Partition Size**: 150 KB
- **Challenge**: Fitting a 200 KB assembler into a 150 KB partition is impossible without overlays.

9.10.3 Overlay Technique: Solution to Memory Constraints

9.10.3.1 How Overlays Work

1. **Program Division**:
    - The program is divided into logical modules called **overlays**.
    - Each overlay represents a distinct portion of the program's functionality.
2. **Initial Loading**:
    - Only the **primary overlay** is loaded into memory along with the main program code.
    - The primary overlay contains essential code to start the program and manage overlay swapping.

9.10.3.2 Overlay Management

- **Overlay Driver**:
    - A specialized module responsible for managing overlays.
    - **Functionality**:
        - When the program requires a module not currently in memory, the overlay driver:
            1. Swaps out the currently loaded overlay.
            2. Replaces it with the required overlay from secondary storage (e.g., disk).
        - Transfers control back to the appropriate point in the program after swapping.

9.10.4 Applying Overlays to the Assembler Example

9.10.4.1 Overlay Design

- **Observation**: Pass 1 and Pass 2 codes are not needed simultaneously.
- **Solution**: Create two overlays:
    - **Overlay A**:
        - Contains:
            - Pass 1 code (70 KB)
            - Symbol table (25 KB)

        - Common routines (25 KB)
        - Overlay driver
      - **Total Size**: 130 KB
  - **Overlay B**:
    - Contains:
      - Pass 2 code (80 KB)
      - Symbol table (25 KB)
      - Common routines (25 KB)
      - Overlay driver
    - **Total Size**: 140 KB
- **Advantage**: Both overlays fit within the 150 KB partition limit.

9.10.4.2 Execution Flow with Overlays

1. **Initial State**:
   - Overlay A is loaded into memory by the overlay driver.
2. **After Pass 1 Completion**:
   - Overlay driver is invoked.
   - Overlay A is swapped out.
   - Overlay B is loaded into memory, overwriting Overlay A.
   - Control is transferred to Pass 2 for execution.

9.10.5 Benefits of Overlays

9.10.5.1 Efficient Memory Usage

- **Minimal Memory Footprint**:
  - Only necessary parts of the program are loaded into memory at any given time.
  - Reduces memory wastage.
- **Performance Improvement**:
  - Reduces disk I/O and paging overhead by keeping only essential program parts in memory.

9.10.5.2 Historical and Modern Relevance

- **Historical Context**:
  - Common in older systems with limited memory (e.g., early mainframe computers, early personal computers).
- **Modern Context**:
  - Less necessary in systems with large memory capacities and sophisticated virtual memory.
  - **Current Relevance**: Still used in certain **embedded systems** where memory constraints persist.

9.10.6 Summary of Key Learnings

1. **Problem Addressed**: Loading a process larger than the main memory partition size.
2. **Solution**: Overlay technique, demonstrated using an assembler example.
3. **Advantages**:
   - Efficient memory usage.
   - Minimal memory footprint.
   - Improved performance by reducing disk I/O and paging overhead.
4. **Relevance**:
   - Historically significant in systems with limited memory.
   - Currently relevant in embedded systems.

9.11 Paging - Examples

9.11.1 Problem 1: Logical and Physical Address Bits Calculation

9.11.1.1 Problem Statement

- **Given:**
  - Logical address space consists of **64 pages**.
  - Each page size = **1,024 words**.
  - Physical memory consists of **32 frames**.
- **Objective:**
  - Determine the number of bits required for **logical addresses**.
  - Determine the number of bits required for **physical addresses**.

9.11.1.2 Solution for Part (a): Number of Bits in Logical Address

9.11.1.2.1 Step 1: Calculate Logical Memory Size

- **Formula:**

  Logical Memory Size = Number of Pages x Page Size

- **Substitution:**

  64 pages x 1,024 words/page = 65,536 words = 64 KB (Kilowords)

9.11.1.2.2 Step 2: Determine Number of Bits (n) for Logical Address

- **Formula:**

  2^n = Logical Memory Size in Words

- **Substitution:**

  2^n = 65,536 => n = 16 bits

- **Conclusion:**
  - **16 bits** are required to represent a logical address in this system.

9.11.1.3 Solution for Part (b): Number of Bits in Physical Address

9.11.1.3.1 Step 1: Calculate Physical Memory Size

- **Assumption:**
  - Frame size = Page size = **1,024 words**.
- **Formula:**

  Physical Memory Size = Number of Frames x Frame Size

- **Substitution:**

  32 frames x 1,024 words/frame = 32,768 words = 32 KB (Kilowords)

9.11.1.3.2 Step 2: Determine Number of Bits (m) for Physical Address

- **Formula:**

  2^m = Physical Memory Size in Words

- **Substitution:**

  2^m = 32,768 => m = 15 bits

- **Conclusion:**
  - **15 bits** are required to represent a physical address in this system.

9.11.2 Problem 2: Page Number and Offset Calculation

9.11.2.1 Problem Statement

- **Given:**
  - Page size = **1 KB (Kilobyte)**.
  - Logical address = **3085 (decimal)**.
- **Objective:**
  - Determine the **page number** and **offset** for the given address.

9.11.2.2 Solution

9.11.2.2.1 Step 1: Convert Logical Address to Binary

- **Decimal Address:** 3085
- **Binary Equivalent:**

  3085_{10} = 110000001101_2

9.11.2.2.2 Step 2: Determine Number of Bits for Offset (d)

- **Given:**
  - Page size = 1 KB = 2^{10} bytes (since 1 KB = 1,024 bytes).
- **Formula:**

  2^d = Page Size => d = 10 bits

- **Interpretation:**
  - The **least significant 10 bits** of the address represent the **offset**.
  - The **remaining bits** represent the **page number**.

9.11.2.2.3 Step 3: Extract Offset and Page Number

- **Binary Address:** `11 0000001101` (split into page number and offset)
  - **Offset (last 10 bits):** `0000001101` = 13_{10}
  - **Page Number (remaining bits):** `11` = 3_{10}

9.11.2.2.4 Final Answer:

- **Page Number:** 3
- **Offset:** 13
- **Interpretation:**
  - The logical address **3085** refers to **page 3, offset 13**.

9.11.3 Key Concepts Reinforced

1. **Logical Address Space:**
   - Defined by the number of pages and page size.
   - Total size = Number of Pages × Page Size.
   - Number of bits required = log_2(Total Words).
2. **Physical Address Space:**
   - Defined by the number of frames and frame size (equal to page size).
   - Total size = Number of Frames × Frame Size.
   - Number of bits required = log_2(Total Words).
3. **Page Number and Offset Calculation:**
   - **Offset bits (d):** Determined by page size (2^d = Page Size).
   - **Page number:** Remaining bits after extracting the offset.
   - **Example:** For a 1 KB page, the offset is **10 bits**, and the rest form the page number.

9.12 Segmentation - Example

9.12.1 Introduction to Segmentation and Address Translation

- This lecture focuses on **segmentation**, a memory management technique in operating systems.
- The primary objective is to demonstrate how **logical addresses** (generated by a process) are translated into **physical addresses** (actual memory locations) using a **segment table**.

9.12.2 Segment Table Structure

- A **segment table** is a data structure used by the operating system to manage memory segments.
- The table is **indexed by segment number (s)**.
- Each entry in the segment table contains two key fields:
  1. **Base (starting physical address)** of the segment in memory.
  2. **Limit (size)** of the segment, defining the maximum valid offset within the segment.

9.12.2.1 Example Segment Table

Segment Number (s)	Base	Limit
0	128	512
1	8,192	2,048
…	…	…

9.12.3 Logical to Physical Address Translation Process

- A **logical address** is represented as a pair: **(s, d)**, where:
  - **s**: Segment number (index into the segment table).
  - **d**: Offset (displacement) within the segment.
- The translation process involves the following steps:

9.12.3.1 Step 1: Retrieve Base and Limit

- Use the segment number **s** to index into the segment table.
- Retrieve the corresponding **base** and **limit** values for the segment.

9.12.3.2 Step 2: Validate the Offset

- Compare the offset **d** with the **limit** of the segment.
  - If **d < limit**, the offset is **valid** (within the segment's bounds).
  - If **d >= limit**, the offset is **invalid**, resulting in a **segmentation fault**.

9.12.3.3 Step 3: Calculate Physical Address

- If the offset is valid, compute the **physical address** as:

```
Physical Address = Base + Offset (d)
```

9.12.4 Example 1: Valid Address Translation

9.12.4.1 Logical Address: (0, 430)

1. **Segment Number (s)**: 0
   - Retrieve **base = 128** and **limit = 512** from the segment table.
2. **Offset (d)**: 430
   - Check if **430 < 512** -> **True** (offset is within bounds).
3. **Physical Address Calculation**:

```
Physical Address = 128 (base) + 430 (offset) = 558
```

4. **Result**: The physical address **558** is valid.

9.12.5 Example 2: Invalid Address Translation (Segmentation Fault)

9.12.5.1 Logical Address: (1, 2056)

1. **Segment Number (s)**: 1
   - Retrieve **base = 8,192** and **limit = 2,048** from the segment table.
2. **Offset (d)**: 2,056
   - Check if **2,056 < 2,048** -> **False** (offset exceeds segment limit).
3. **Result**:
   - The offset **2,056** is **invalid** because it exceeds the segment's limit of **2,048**.
   - This triggers a **segmentation fault**, indicating an attempt to access memory outside the allocated segment.

9.12.6 Key Observations

1. **Segmentation Fault**:
   - Occurs when a process attempts to access an offset beyond the segment's limit.
   - The operating system terminates the process or handles the fault (e.g., by allocating more memory or terminating the program).
2. **Valid Address Range**:
   - For a segment with **base = B** and **limit = L**, valid physical addresses range from **B** to **B + L - 1**.

3. **Protection**:
    - The **limit** ensures that a process cannot access memory outside its allocated segment, providing **memory protection**.

9.12.7 Summary of Steps for Address Translation

1. **Extract segment number (s) and offset (d)** from the logical address.
2. **Look up the segment table** to find the **base** and **limit** for segment **s**.
3. **Compare the offset (d) with the limit**:
    - If **d < limit**, proceed to calculate the physical address.
    - If **d >= limit**, raise a **segmentation fault**.
4. **Compute the physical address** as **base + d** (if valid).

9.12.8 Practical Implications

- **Efficiency**: Segment tables enable efficient memory access by reducing the need for contiguous memory allocation.
- **Flexibility**: Segments can grow or shrink dynamically (within their limits), accommodating varying memory requirements.
- **Security**: Limits prevent processes from accessing unauthorized memory regions, enhancing system stability and security.

9.12.9 Conclusion

- The segment table is a critical component in **segmentation-based memory management**, facilitating the translation of logical to physical addresses while enforcing memory protection.
- Understanding this process is essential for designing and debugging operating systems, particularly in memory management modules.

9.13 Summary

9.13.1 Introduction to Main Memory Management

- The **operating system (OS)** plays a **critical role** in managing **main memory (RAM)**.
- Key functions performed by the OS in memory management include:
    - **Memory allocation**
    - **Memory deallocation**
    - **Memory protection**
    - **Memory paging and segmentation**
    - **Memory scheduling**

9.13.2 Key Responsibilities of the OS in Memory Management

9.13.2.1 1. Memory Allocation

- The OS assigns **memory space** to **processes** when they are loaded into RAM for execution.
- Ensures that each process has sufficient memory to operate.

9.13.2.2 2. Memory Deallocation

- The OS **reclaims memory** from processes that have **finished execution**.
- Prevents memory leaks and ensures efficient reuse of available memory.

9.13.2.3 3. Memory Protection

- Ensures that **processes do not interfere** with each other's memory.
- Prevents unauthorized access or modification of memory regions.
- Implemented via hardware mechanisms (e.g., **Memory Management Unit (MMU)**).

9.13.2.4 4. Memory Sharing

- Allows **multiple processes** to **share memory** when necessary.
- Useful for **inter-process communication (IPC)** and **efficient resource utilization**.

9.13.2.5 5. Memory Organization

- Involves managing the **memory hierarchy** (e.g., RAM, cache, disk).
- Maintains **memory structures** (e.g., page tables, segment tables) for efficient access.

9.13.3 Memory Allocation Techniques

9.13.3.1 1. Contiguous Allocation

- Assigns a **single contiguous block** of memory to a process.
- **Advantages:**
  - Simple to implement.
  - Low overhead in address translation.
- **Disadvantages:**
  - Can lead to **external fragmentation** (unused gaps between allocated blocks).
  - Requires **compaction** (defragmentation) to reclaim fragmented space.

9.13.3.2 2. Paging

- Divides **physical memory** into fixed-size blocks called **frames**.
- Divides **logical memory (process address space)** into fixed-size blocks called **pages**.
- **Key Features:**
  - Eliminates the need for **contiguous allocation**.
  - Uses a **page table** to map **logical pages** to **physical frames**.
  - Reduces **external fragmentation** (but may introduce **internal fragmentation** if a process does not use an entire page).
- **Address Translation:**
  - The **MMU (Memory Management Unit)** translates **logical addresses** to **physical addresses** using the page table.

9.13.3.3 3. Segmentation

- Divides memory into **logical segments** based on program structure (e.g., functions, arrays, objects).
- **Key Features:**
  - Each segment can **grow or shrink independently**.
  - Uses a **segment table** to map **logical segments** to **physical memory locations**.
  - Supports **dynamic memory allocation** (e.g., stack and heap growth).
- **Advantages:**
  - More **intuitive** for programmers (aligns with program structure).
  - Easier to manage **shared libraries** and **modular code**.
- **Disadvantages:**

- Can lead to **external fragmentation** (similar to contiguous allocation).
- More complex **address translation** compared to paging.

9.13.4 Role of the Memory Management Unit (MMU)

9.13.4.1 1. Hardware Component for Memory Operations

- The **MMU** is a **hardware component** that works with the CPU to manage memory access.
- **Primary Functions:**
  - **Address Translation:**
    - Converts **logical (virtual) addresses** to **physical addresses**.
    - Uses **page tables** (for paging) or **segment tables** (for segmentation).
  - **Memory Protection:**
    - Ensures **process isolation** by preventing unauthorized memory access.
    - Uses **base and limit registers** to define valid memory ranges for a process.

9.13.4.2 2. Base and Limit Registers

- **Base Register:**
  - Stores the **starting address** of a process's memory segment.
- **Limit Register:**
  - Stores the **size (limit)** of the memory segment.
- **Function:**
  - The MMU checks if a memory access falls within the **base + limit** range.
  - If not, a **memory protection fault** (e.g., segmentation fault) is triggered.

9.13.5 Comparison of Memory Management Techniques

Technique	Allocation Unit	Fragmentation	Address Translation	Flexibility
Contiguous	Single block	External	Simple (direct mapping)	Low (fixed allocation)
Paging	Fixed-size pages/frames	Internal (minor)	Complex (page tables)	High (non-contiguous)
Segmentation	Logical segments	External	Complex (segment tables)	High (dynamic growth)

9.13.6 Key Takeaways

1. The OS manages **memory allocation, deallocation, protection, sharing, and organization**.
2. **Contiguous allocation** is simple but suffers from **external fragmentation**.
3. **Paging** eliminates contiguous allocation needs but may introduce **internal fragmentation**.
4. **Segmentation** aligns with program structure but can lead to **external fragmentation**.
5. The **MMU** plays a crucial role in **address translation** and **memory protection** using **base and limit registers**.
6. **Page tables** (for paging) and **segment tables** (for segmentation) are essential for **mapping logical to physical memory**.